Technical Document - "Definitely Spaceship Pilot Certified”
Jack P, Jack G, Abbi W

Algorithms and Calculations

Spaceship Model
For the spaceship model, we took a more simple and blocky approach to its structure, rotating certain parts for added detail and to really get the interior shape, all within a hierarchy within a primary shipGroup node. These sub-groups are floorGroup, dashGroup, structureGroup, frameGroup, and controlGroup, which all just help with the organization process. Speaking of organization, the SpaceshipModel.js uses pre-determined materials and “this” to help cut down on the number of lines, and even then the file itself is quite lengthy.

Rear-view Camera
The rear-view camera uses a dual-camera hierarchy with multi-pass WebGL rasterization and Scissor Testing. a second perspective camera named (rearCamera) is positioned at the origin, and inherits the ship’s transformations to rotate around and look behind the ship. This is continuously rendered onto the screen, taking into account the window’s dimensions, uses scissor testing, and positions at the top middle of the screen.

Asteroids
The asteroids are created using Three.js's InstancedMesh to efficiently render 500 instances of a shared geometry. Each asteroid has randomized positions, sizes, colors, and movement patterns, with continuous animation and repositioning to simulate an infinite asteroid field.

Collision Detection
Collision detection is implemented using Three.js's Box3 to create a bounding box around the spaceship, while each asteroid is represented by a spherical collision radius. The system accounts for each asteroid's animated movement before calculating its distance from the closest point on the spaceship's bounding box. When a collision occurs, a collision normal is calculated to determine the direction and distance needed to push the asteroid away. The asteroid's velocity is then adjusted using vector calculations to simulate a bouncing effect, preventing it from remaining inside the spaceship.

Textures
Textures are made with a custom GLSL shader, and can be found on various little “screens” within the ship. These screens have procedural gridlines on them, with dynamic colors and tile spacing to make them stand out and have a bit of a high-tech feel to them. The asteroids also use custom GLSL shaders to generate procedural rocky textures, using noise functions to create unique surface

patterns and variations in color. These textures are also animated over time to give the asteroids a more dynamic appearance.

Above & Beyond Requirement - The Booster

Our above and beyond was that implementation of a boost. When the user holds down Mouse 1, the ship speed gradually increases to a variable amount, and FOV increases as well. Upon releasing, the two values return to normal. This was originally implemented for debugging asteroid collision, as flying slowly towards small objects that moved faster and more erratically was proving quite troublesome.

Challenges We Faced

Timing
Ended up underestimating how much time it would take to model and really piece together the whole project, combined with a busy week for all of us.

GitHub
GitHub went smoothly this time around, and ideally it stays that way for the future :)

Debugging
Aspects of the physics interactions between the objects were challenging, as many implementations did not function as expected. This required a lot of debugging to make movements more regular and smooth.